Módulo 01: Build and Run Container Images with ACR Tasks

Fundamentos de Azure CLI, ciclo de vida serverless OCI y gobernanza en Azure Container Registry

PRERREQUISITOS UNIVERSALES DEL ENTORNO

- **Suscripción de Azure:** Requiere plan con soporte de ACR Tasks (ej. `Pay-As-You-Go`).
- **Azure CLI:** Versión 2.50+ instalada localmente o en Azure Cloud Shell (`az --version`).
- **Python 3.10+:** Para ejecutar scripts de despliegue automatizado (`azdeploy.py`).
- **Permisos RBAC:** Rol `Contributor` sobre el Resource Group o `AcrPush` / `AcrPull`.
- **Sin Docker local:** No se requiere Docker Desktop ni daemon de contenedores instalado.

CONSIDERACIONES Y AVISOS CRÍTICOS DE AZURE

- **Restricción de suscripción:** ACR Tasks no está disponible en suscripciones con créditos gratuitos.
- **Unicidad global de DNS:** El nombre de ACR debe ser único a nivel mundial (`<name>.azurecr.io`).
- **Latencia de propagación de CLI:** Al crear registros, reintente los comandos ante demoras de replicación.
- **Sintaxis de terminal:** PowerShell usa acento grave (`` ` ``) para multilínea; Bash/macOS usa barra invertida (`\`).

Estructura del Proyecto Fuente: Paquete oficial descargado en `CODE/1. acr-tasks-python/` que contiene la aplicación web Flask (`api/app.py`), el archivo de construcción OCI (`api/Dockerfile`) y el script de aprovisionamiento de infraestructura (`azdeploy.py`).

FUNDAMENTOS Y ANATOMÍA DE AZURE CLI (GUÍA PARA NUEVOS USUARIOS)

Estructura jerárquica de comandos: Toda instrucción sigue el patrón: `az <servicio> <subgrupo> <acción> [--parámetros]`. Por ejemplo: `az acr repository list` interactúa con el servicio ACR, subgrupo repositorio, y la acción listar.

Parámetro / Flag	Propósito Técnico	Ejemplo de Uso
`--resource-group` (`-g`)	Define el contenedor lógico de ciclo de vida en Azure.	`--resource-group rg-lab-eastus`
`--name` (`-n`)	Nombre del recurso que se está creando o consultando.	`--name acra1b2c3d4`
`--registry`	Nombre del Azure Container Registry destino en ACR Tasks.	`--registry $env:ACR_NAME`
`--image`	Repositorio y etiqueta asignada a la imagen compilada.	`--image inference-api:v1.0.0`
`--output` (`-o`)	Formato de salida: `table` (legible), `json` o `tsv` (parseo limpio).	`--output table` | `-o tsv`
`--query` (JMESPath)	Filtra la respuesta en el cliente para extraer un dato específico.	`--query "id"` | `--query "registrationState"`
`--no-wait` / `--yes`	Ejecuta la operación de forma asíncrona sin pedir confirmación.	`az group delete ... --no-wait --yes`

FASE 1: PREPARACIÓN DEL ENTORNO, IDENTIDAD Y PROVEEDORES

`az login & az provider register`

IDENTIDAD & API

Inicie sesión en su cuenta de Azure y registre el proveedor de recursos de contenedores en la suscripción:

```
# 1. Autenticación interactiva en la CLI de Azure
az login

# 2. Verificar la suscripción activa actual
az account show --output table

# 3. Registrar el Resource Provider de Azure Container Registry (obligatorio)
az provider register --namespace Microsoft.ContainerRegistry

# 4. Verificar el estado del proveedor (debe retornar "Registered")
az provider show `
  --namespace Microsoft.ContainerRegistry `
  --query "registrationState" `
  -o tsv
```

¿Por qué registrar el proveedor?: Las suscripciones de Azure requieren registrar explícitamente el espacio de nombres `Microsoft.ContainerRegistry` antes de poder aprovisionar registros y ejecutar tareas de compilación serverless.

Desglose Técnico del Código del Laboratorio

Análisis arquitectónico de azdeploy.py, Dockerfile y la API de inferencia Flask (CODE/1. acr-tasks-python)

ANÁLISIS ARQUITECTÓNICO DE LOS ARCHIVOS DEL PROYECTO

1. azdeploy.py (Automatización de Infraestructura)

INFRAESTRUCTURA COMO CÓDIGO

Script en Python que orquesta el aprovisionamiento de recursos en Azure sin requerir plantillas Bicep ni Terraform:

- **Variables de entrada (L5-6):** Define `rg` (Resource Group) y `location` (región). Son los únicos parámetros editables.
- **Resolución agnóstica de CLI (L25-35):** `_resolve_exe("az")` ubica el binario `az` en Linux/macOS o `az.cmd` en Windows.
- **Identidad y Hashing único (L60-68, L104-106):** Ejecuta `az ad signed-in-user show --query id -o tsv` para obtener el Object ID de Entra ID. Calcula un hash SHA-1 de 8 caracteres (`user_hash`) y genera el nombre `acr{user_hash}`. *Motivo:* Los nombres de ACR son FQDN globales (`<name>.azurecr.io`) y deben ser únicos en todo el mundo.
- **Idempotencia en Resource Group (L120-137):** Consulta `az group exists`; si no existe, lo crea con `az group create`.
- **Aprovisionamiento del ACR (L139-154):** Ejecuta `az acr create --sku Basic` para crear el registro de bajo costo.
- **Exportación Multi-Shell (L71-102):** Escribe `.env` (para Bash: `export VAR="..."`) y `.env.ps1` (para PowerShell: `$env:VAR = "..."`) con UTF-8 y saltos LF, garantizando compatibilidad entre terminales.

2. api/Dockerfile (Definición de Contenedor OCI)

CONTENEDOR OCI

Especificación de capas para empaquetar el microservicio de inferencia de IA en una imagen estándar:

- `FROM python:3.11-slim`: Imagen base oficial de Python sobre Debian mínimo (~150 MB). Reduce la superficie de ataque y el tiempo de transferencia en comparación con imágenes completas.
- `WORKDIR /app`: Establece el directorio de trabajo aislado dentro del sistema de archivos del contenedor.
- `RUN pip install --no-cache-dir flask`: Instala el microframework web. El flag `--no-cache-dir` evita almacenar archivos temporales de pip, manteniendo la capa liviana.
- `COPY app.py .`: Copia el script del microservicio al directorio de trabajo `/app`.
- `ENV APP_VERSION=1.0.0` y `ENV MODEL_VERSION=v1`: Inyecta variables de entorno predeterminadas, desacoplando la configuración del código fuente.
- `EXPOSE 5000` y `CMD ["python", "app.py"]`: Declara el puerto de escucha y el comando de arranque del servidor.

3. api/app.py (Microservicio de Inferencia de IA)

MICROSERVICIO REST

Servidor HTTP REST implementado con Flask que simula una API de inferencia de Machine Learning:

- **Ruta `/health`:** Health check probe. Retorna `{"status": "healthy", "version": ...}`. Esencial para que orquestadores (AKS, Azure Container Apps) comprueben la salud (liveness y readiness).
- **Ruta `/predict`:** Simulación de inferencia. Retorna predicción, nivel de confianza (`0.95`) y la versión del modelo leída dinámicamente de la variable de entorno `MODEL_VERSION`.
- **Ruta `/`:** Catálogo de endpoints disponibles (`/health`, `/predict`) para autodescubrimiento.
- **Arranque (`host='0.0.0.0', port=5000`):** Vincular el servidor a `0.0.0.0` es mandatorio para permitir conexiones provenientes del exterior del contenedor.

FASE 1 (PARTE B): EJECUCIÓN DEL DESPLIEGUE Y CARGA DE VARIABLES

python azdeploy.py & Carga de Entorno

AUTOMATIZACIÓN

```
# 1. Configurar variables iniciales en azdeploy.py (ej. rg="rg-ai-lab", location="eastus")
# 2. Ejecutar el script desde el directorio raíz del proyecto (CODE/1. acr-tasks-python)
python azdeploy.py

# 3. Cargar las variables de entorno generadas en la sesión actual de la terminal:
# En PowerShell:
. .\.env.ps1

# En Linux / macOS / Bash:
source .env

# 4. Validar que la variable de entorno contiene el nombre del registro
echo $env:ACR_NAME  # En PowerShell
echo $ACR_NAME      # En Bash/Zsh
```

Estructura de ACR, Niveles de Servicio y Desduplicación

Jerarquía OCI, espacios de nombres, artefactos Helm y tiers de escalabilidad (Contenido Oficial PPT)

ORGANIZACIÓN Y ALMACENAMIENTO EN AZURE CONTAINER REGISTRY

1. JERARQUÍA DEL REGISTRO

- **Registro:** Servicio privado administrado en Azure (`<acr>.azurecr.io`).
- **Repositorio:** Agrupación lógica de imágenes con el mismo nombre (ej. `inference-api`).
- **Artefactos:** Imágenes Docker, gráficos de **Helm** y contenido compatible con OCI.

2. ESPACIOS DE NOMBRES

- **Organización Multinivel:** ACR permite estructurar repositorios con barras diagonales (`/`).
- **Ejemplo corporativo:** `myacr.azurecr.io/marketing/campaign:v1` .
- **Gobierno RBAC:** Facilita asignar permisos basados en prefijos de repositorios.

3. DESDUPLICACIÓN DE CAPAS

- **Direccionable por contenido:** Las capas OCI se indexan por su hash SHA-256 interno.
- **Almacenamiento único:** Si 50 imágenes usan `python:3.11-slim` , la capa se almacena **una sola vez**.
- **Ahorro masivo:** Minimiza costos de almacenamiento y tiempo de push/pull.

NIVELES DE SERVICIO (SKUS) DE AZURE CONTAINER REGISTRY

Nivel (SKU)	Capacidades y Rendimiento	Escenario de Uso Recomendado
Básico (Basic)	10 GB de almacenamiento incluido. Límites de rendimiento equilibrados para entornos no productivos. Sin Private Link.	Desarrollo individual, laboratorios de certificación y prototipos iniciales.
Estándar (Standard)	100 GB de almacenamiento. Mayor rendimiento de lectura/escritura (operaciones por minuto) y mayor tasa de transferencia.	Cargas de trabajo de producción estándar con despliegues frecuentes y pipelines CI/CD.
Premium	500 GB de almacenamiento. **Replicación Geográfica** multi-región, Content Trust (firmado digital con Notary), Private Endpoints (VNet) y retención.	Arquitecturas globales multirregión (AKS multi-cluster), alta disponibilidad y máxima seguridad empresarial.

FASE 2: COMPILACIÓN CLOUD SERVERLESS CON ACR TASKS (QUICK TASKS)

`az acr build` (Tarea 2 del Laboratorio)

SERVERLESS BUILD

Empaqueta el contexto local, lo transfiere por HTTPS a Azure y ejecuta la compilación en agentes administrados sin requerir Docker instalado localmente:

```
# Compilación básica y push directo al registro usando el contexto local ./api
az acr build `
  --registry $env:ACR_NAME `
  --image inference-api:v1.0.0 `
  ./api

# Desglose de flags:
# --registry : Especifica el ACR que recibirá la imagen una vez compilada.
# --image    : Nombre del repositorio y tag a asignar (inference-api con tag v1.0.0).
# ./api      : Carpeta local con el Dockerfile y app.py comprimida y subida como contexto.

# Variaciones avanzadas de compilación en la nube:
# 1. Especificar un Dockerfile alternativo:
az acr build --registry $env:ACR_NAME --image inference-api:v1.0.0 --file ./api/Dockerfile.prod ./api

# 2. Inyección de argumentos de construcción en tiempo de compilación:
az acr build --registry $env:ACR_NAME --image inference-api:v1.0.0 --build-arg MODEL_VERSION=2.1 ./api
```

Ciclo de Vida de ACR Tasks y Validación On-Demand

Las 5 fases de build en Azure, inspección OCI, direccionamiento y testing serverless con az acr run

Ciclo de Vida Interno de ACR Tasks (5 Fases en la Nube de Azure):
1. **Empaquetado y Carga:** Comprime el contexto de `./api` en un tarball y lo sube por HTTPS seguro al storage de ACR.
2. **Encolado de Tarea:** Encola la solicitud y aprovisiona un agente de cómputo efímero y aislado administrado por Azure.
3. **Streaming de Capas:** Ejecuta las instrucciones del Dockerfile transmitiendo los logs en vivo a su terminal.
4. **Inserción en Registro:** Publica la imagen terminada directamente en su Azure Container Registry privado.
5. **Reporte de Digest y Estado:** Genera el hash inmutable SHA-256 y confirma el resultado exitoso (*Task Succeeded*).

FASE 3: INSPECCIÓN DEL REGISTRO, REPOSITORIOS E INMUTABILIDAD OCI

`az acr repository & az acr manifest (Tarea 3)`

INSPECCIÓN OCI

Confirme que el artefacto existe en el registro mediante consultas a repositorios, etiquetas y metadatos:

```
# 1. Listar repositorios en el ACR (confirma la presencia de inference-api)
az acr repository list --name $env:ACR_NAME --output table

# 2. Listar todas las etiquetas asignadas al repositorio inference-api
az acr repository show-tags `
  --name $env:ACR_NAME `
  --repository inference-api `
  --output table

# 3. Extraer metadatos detallados y el hash criptográfico inmutable (Digest SHA-256)
az acr manifest list-metadata `
  --registry $env:ACR_NAME `
  --name inference-api `
  --output table
```

DIRECCIONAMIENTO DE IMÁGENES OCI: ETIQUETAS VS RESÚMENES (PPT SLIDE 6)

Método de Referencia	Sintaxis Técnica Oficial	Comportamiento en Producción
Por Etiqueta (Tag)	`registry.azurecr.io/api:v1.2.0`	Puntero legible pero **mutable**. Un push posterior puede reasignar el tag y provocar discrepancias entre nodos.
Por Resumen (Digest SHA-256)	`registry.azurecr.io/api@sha256:0a2e...`	Identificador **estrictamente inmutable**. Garantiza que todos los clústeres y nodos ejecuten la versión idéntica.

FASE 4: EJECUCIÓN ON-DEMAND DE LA IMAGEN CON AZ ACR RUN (TESTING CLOUD)

`az acr run (Tarea 4 del Laboratorio)`

TESTING CLOUD

Instancia un contenedor efímero serverless para validar que la API Flask (`app.py`) carga sin errores y sin desplegar clústeres:

```
# Validar la importación del framework Flask y los módulos de inferencia sin contexto
az acr run `
  --registry $env:ACR_NAME `
  --cmd "$env:ACR_NAME.azurecr.io/inference-api:v1.0.0 python -c 'from app import app'" `
  /dev/null

# Desglose de parámetros:
# --cmd     : Comando a ejecutar dentro del contenedor una vez descargado.
# /dev/null : Indica que no se transfiere contexto de archivos local (ejecución pura de la imagen).
```

Criterio de validación: La consola muestra el progreso del *Docker pull*. Una ejecución exitosa culmina con `Run ID: xxx was successful after xxx`, garantizando que el entorno Python y sus módulos arrancan correctamente.

Automatización CI/CD, Multiversioning y Gobernanza

Disparadores por Git Commit, macros dinámicas, bloqueo inmutable y purga programada con acr purge

AUTOMATIZACIÓN AVANZADA: DESENCADENADORES DE CÓDIGO FUENTE (PPT SLIDE 7)

az acr task create (Source Trigger en GitHub)

AUTOMATIZACIÓN CI

Crea una tarea persistente en Azure que compila automáticamente la imagen cada vez que se realiza un commit en el repositorio:

```
# Crear tarea automatizada con disparador por Git commit y token de acceso personal (PAT)
az acr task create `
  --registry $env:ACR_NAME `
  --name build-inference-api `
  --image inference-api:{{.Run.ID}} `
  --context https://github.com/myorg/inference-api.git#main `
  --file Dockerfile `
  --git-access-token $PAT
```

Macros de variables dinámicas de ACR Tasks: El parámetro `{{.Run.ID}}` asigna un identificador único correlativo por compilación. Otras macros útiles: `{{.Run.Git.SHA}}` (commit hash de Git) y `{{.Run.Date}}` (marca de tiempo).

FASE 5: MULTIVERSIONING DE CONTENEDORES CON NUEVA ETIQUETA (TAREA 5)

az acr build v1.1.0 & show-tags

MULTIVERSIÓN

```
# Compilar y subir la nueva iteración de la imagen con la etiqueta v1.1.0
az acr build --registry $env:ACR_NAME --image inference-api:v1.1.0 ./api

# Listar etiquetas para verificar la coexistencia de ambas versiones en el registro
az acr repository show-tags --name $env:ACR_NAME --repository inference-api --output table
```

FASE 6: GOBERNANZA, BLOQUEO DE IMAGEN Y PURGA DE IMÁGENES HUÉRFANAS

Bloqueo de Producción

INMUTABILIDAD

Bloquea la imagen para evitar eliminación o sobreescritura accidental:

```
# Impedir sobreescritura de v1.0.0
az acr repository update `
  --name $env:ACR_NAME `
  --image inference-api:v1.0.0 `
  --write-enabled false

# Impedir eliminación
az acr repository update `
  --name $env:ACR_NAME `
  --image inference-api:v1.0.0 `
  --delete-enabled false
```

Purga Programada (acr purge)

AHORRO COSTES

Programa la limpieza semanal de imágenes sin etiqueta (dangling):

```
# Tarea programada semanalmente
az acr task create `
  --registry $env:ACR_NAME `
  --name cleanup-untagged `
  --cmd "acr purge --filter '.*:.*' --untagged --ago 7d" `
  --schedule "0 0 * * 0" `
  --context /dev/null
```

Limpieza de Recursos y Diagnóstico Oficial

Desaprovisionamiento asíncrono, chequeo de salud y resolución de incidentes frecuentes en Azure

FASE 7: LIMPIEZA DE RECURSOS Y COMANDOS DE DIAGNÓSTICO

`az group delete & az acr check-health`

CLEAN UP

Comandos para finalizar el laboratorio y eliminar recursos para evitar consumos no deseados:

```
# Borrado seguro y asíncrono del Resource Group y todos los recursos anidados
az group delete --name <your-resource-group-name> --no-wait --yes

# Comandos complementarios de depuración y acceso local:
# Diagnostica conectividad de red, resolución DNS y servicio Docker
az acr check-health --name $env:ACR_NAME --yes

# Autentica el cliente Docker local con el registro privado de Azure (si se utiliza localmente)
az acr login --name $env:ACR_NAME
```

⚠️ **Precaución de Limpieza en Cascada:** El comando `az group delete` elimina de forma permanente el grupo de recursos y todos los componentes asociados (registro ACR, repositorios, etiquetas e historiales de tareas). Si utilizó un grupo compartido, asegúrese de que no contenga otros recursos en producción.

GUÍA OFICIAL DE RESOLUCIÓN DE PROBLEMAS (TROUBLESHOOTING)

1. SESIÓN Y VARIABLES DE ENTORNO

- **Verificar suscripción:** Ejecute `az account show` para confirmar que opera en la suscripción activa adecuada.
- **Variable vacía:** Si `$env:ACR_NAME` está vacía en una terminal nueva, reejecute: `. .\.env.ps1` (PowerShell) o `source .env` (Bash).
- **Suscripción Pay-As-You-Go:** Recuerde que ACR Tasks requiere plan de pago habilitado.

2. DESPLIEGUE Y FALLOS DE COMPILACIÓN

- **Estado del ACR:** Verifique en portal o ejecute `az acr list --output table`; el *Provisioning State* debe ser **Succeeded**.
- **Ruta de compilación:** Confirme que ejecuta la orden desde la raíz del proyecto donde reside la carpeta `./api` con su `Dockerfile`.
- **Diagnóstico de tareas:** Inspeccione errores con `az acr task list-runs --registry $env:ACR_NAME --output table`.

REGLAS DE ORO DE ARQUITECTURA PARA EL EXAMEN (PPT SLIDE 12)

REGLA 1: INMUTABILIDAD POR MANIFIESTO

En producción, referencie siempre la imagen por su **SHA-256 Digest** y aplique `--write-enabled false` para impedir modificaciones accidentales.

REGLA 2: SIN DOCKER LOCAL

Cuando los desarrolladores o runners CI/CD carezcan de Docker Daemon o permisos de root, la solución oficial es **ACR Quick Tasks** (`az acr build`).

REGLA 3: BASE IMAGE PATCHING

ACR Tasks permite configurar **Base Image Triggers** para recompilar contenedores automáticamente cuando la imagen base de Python/Linux recibe parches de seguridad.

REGLA 4: OPTIMIZACIÓN CON ACR PURGE

Configure tareas programadas con `acr purge --untagged --ago 7d` para eliminar imágenes huérfanas sin tag y mantener los costos de almacenamiento bajo control.

Preguntas Oficiales de Examen de Certificación

Knowledge check oficial de Microsoft Learn (Slide 10 PPT) y escenarios de arquitectura con justificación técnica

Pregunta 1: Entorno de Compilación Controlado (Oficial PPT Slide 10) — Quick Tasks

El equipo crea imágenes de contenedor en estaciones de trabajo de desarrollador, lo que conduce a resultados incoherentes. Necesitas que todas las imágenes se integren en un entorno controlado en la nube sin depender del hardware local. ¿Qué característica de ACR aborda esto?

A) Replicación geográfica del registro
B) Creación rápida de ACR Tasks (Quick Tasks con `az acr build`)
C) Espacios de nombres del repositorio

✓ **Respuesta Correcta: B) Creación rápida de ACR Tasks (Quick Tasks)**

Justificación Oficial: Las tareas rápidas (Quick Tasks) ejecutan el proceso de construcción y push directamente en agentes efímeros y controlados de Azure, eliminando divergencias causadas por las versiones del daemon de Docker o dependencias locales de cada máquina de desarrollo.

Pregunta 2: Consistencia Estricta en Producción (Oficial PPT Slide 10) — Digest SHA-256

Debes implementar una imagen de contenedor en producción y asegurarte de que cada nodo ejecute exactamente la misma versión, incluso si alguien inserta una nueva imagen con la misma etiqueta. ¿Cómo debes hacer referencia a la imagen en el manifiesto de despliegue?

A) Por la etiqueta `latest`
B) Por hash de manifiesto (Resumen SHA-256 / Digest)
C) Por etiqueta de versión semántica (ej. `:v1.0.0`)

✓ **Respuesta Correcta: B) Por hash de manifiesto (Resumen SHA-256 / Digest)**

Justificación Oficial: Las etiquetas son referencias mutables por definición en la especificación OCI. Para garantizar que cada nodo del clúster descargue el mismo binario exacto sin posibilidad de sobreescritura, se debe referenciar por el hash de manifiesto (`image@sha256:...`).

Pregunta 3: Gobernanza y Bloqueo de Imágenes en ACR — Gobernanza OCI

En un registro de producción se desea bloquear la imagen `inference-api:v1.2.0` para impedir que los pipelines de CI/CD sobrescriban accidentalmente la etiqueta o eliminen el artefacto. ¿Qué comando ejecuta esta política?

A) `az acr repository update --name myacr --image inference-api:v1.2.0 --write-enabled false`
B) `az acr repository delete --name myacr --image inference-api:v1.2.0 --yes`
C) Modificar el SKU del registro de Standard a Basic

✓ **Respuesta Correcta: A) az acr repository update --name myacr --image inference-api:v1.2.0 --write-enabled false**

Justificación: El flag `--write-enabled false` bloquea la imagen contra sobreescritura y eliminación accidental, garantizando la inmutabilidad de la versión en el registro privado.

Pregunta 4: Parcheo Automatizado de Vulnerabilidades en Imágenes Base — Base Image Triggers

Se publica un parche de seguridad crítico (CVE) en la imagen base pública `python:3.11-slim`. Se requiere que todas las imágenes dependientes en ACR se reconstruyan automáticamente sin intervención manual ni pipelines externos. ¿Qué solución se debe implementar?

A) Tareas cron periódicas en Azure Container Instances
B) ACR Tasks con desencadenador de imagen base (Base Image Update Trigger)
C) Webhooks de GitHub invocando un Azure Automation Runbook

✓ **Respuesta Correcta: B) ACR Tasks con desencadenador de imagen base (Base Image Update Trigger)**

Justificación: ACR Tasks detecta de forma nativa cuándo una imagen base (en el propio ACR o en registros públicos como Docker Hub) recibe una actualización y recompila automáticamente las imágenes derivadas dependientes.

Pregunta 5: Restricciones de Nomenclatura y DNS de Azure Container Registry — Arquitectura & DNS

Un administrador intenta aprovisionar un registro con el nombre `My_Prod_ACR_EastUS` mediante la CLI y recibe un error de validación. ¿Cuál es el requisito técnico de Azure que causó el fallo?

A) Debe tener entre 5 y 50 caracteres alfanuméricos exclusivamente y ser único a nivel global en Azure.
B) El SKU Basic únicamente admite nombres que inicien con el prefijo "registry".
C) Los registros de contenedores requieren que el nombre coincida exactamente con el Resource Group.

✓ **Respuesta Correcta: A) Debe tener entre 5 y 50 caracteres alfanuméricos exclusivamente y ser único globalmente.**

Justificación: Cada ACR genera un FQDN público resoluble por DNS (`<nombre>.azurecr.io`). Por diseño en Azure, el nombre solo permite números y letras (sin guiones bajos _ ni mayúsculas) y debe ser único a nivel mundial.